Authentication

Interface Design Description

Eclipse Arrowhead 5.2.2 | core system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-core-java-spring/authentication. 7 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-core-go/authentication.

Authentication - Go 5.2.2

Standalone core system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/Authentication. Entry point: ./cmd/authentication. Module: arrowhead/ah5-core-go/authentication. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8444 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8444". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../../LISTENING-PORTS.md) for precedence and all systems.

Detailed implementation evidence

0 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Go HTTP route inventory

This is the repository route inventory, not a new differential test. Route presence does not establish DTO, error, authorization or persistence equivalence.

- GET /authentication/general/mgmt/get-config | Go route recorded: True

- POST /authentication/general/mgmt/logs | Go route recorded: True

- POST /authentication/identity/change | Go route recorded: True

- POST /authentication/identity/login | Go route recorded: True

- POST /authentication/identity/logout | Go route recorded: True

- GET /authentication/identity/verify/{token} | Go route recorded: True

- POST /authentication/mgmt/identities/query | Go route recorded: True

- DELETE /authentication/mgmt/identities | Go route recorded: True

- POST /authentication/mgmt/identities | Go route recorded: True

- PUT /authentication/mgmt/identities | Go route recorded: True

- DELETE /authentication/mgmt/sessions | Go route recorded: True

- POST /authentication/mgmt/sessions | Go route recorded: True

- GET /authentication/monitor/echo | Go route recorded: True

Native compatibility profile

Compatibility profile and remaining work

This document is deliberately separate from the route inventory. It records the actual implemented behavior and material gaps, rather than calling a set of HTTP handlers a complete Java replacement.

Implemented

- Authentication: PASSWORD identities, bcrypt cost 10, batch management, credential changes, credential-authenticated logout, UUID sessions, one active session per identity, expiration checks, session query/closure and identity query filters. Credentials and session tokens are excluded from management responses. Credential changes and management updates invalidate sessions (a stricter security choice).

- ConsumerAuthorization: provider and MGMT policy levels, MGMT precedence, ALL/WHITELIST/BLACKLIST/SYS_METADATA policies, scoped/default fallback, all-scopes checks, grant/lookup/query/revoke/check operations. Missing policies deny access.

- Authorization tokens: usage/time-limited tokens; URL-base64 self-contained tokens with the Java pipe-separated field order; RS256/RS512 JWTs with psn/csn/ccn/tat/tan/sco claims; public-key export; management generation/query/revocation; provider-bound online verification with atomic usage decrement. Raw tokens are returned only at generation, not retained for later retrieval.

- Encryption-key registration: Java AES/ECB/PKCS5Padding and AES/CBC/PKCS5Padding wire encodings, provider-specific IVs for CBC, owner/management removal. Provider keys are authenticated-encrypted at rest with AES-GCM and provider identity as additional authenticated data. The server certificate's RSA key signs JWTs.

- Simple orchestration: consumer-specific store, unique priorities per consumer/service, priority modification, preferred providers, ONLY_PREFERRED, per-service MATCHMAKING, ignored-field/flag warnings, pull/history.

- Dynamic orchestration: service discovery through ServiceRegistry, mandatory authorization checks, preferred providers, interface operations, token acquisition, MATCHMAKING, provider-consented exclusivity capped by allowExclusivity, atomic lock conflict rechecks, lock management, pull/history.

- Both orchestrators: persisted subscriptions, owner checks, management subscription/query/unsubscribe/trigger operations, persisted jobs and HTTPS push callbacks. Callback TLS is pinned to the target system identity before sending results. Interrupted in-progress jobs become ERROR on restart instead of being redelivered automatically.

- General management: sanitized configuration map, bounded persisted audit log querying, authenticated echo. Source-generated service descriptions and opt-in periodic service registration.

- Shared runtime: mandatory local-cloud mTLS, identity-token certificate binding, fixed authority identity pinning before credentials, redirect rejection, coalesced login refresh, conditional token invalidation, body limits, server timeouts, atomic JSON snapshots and exclusive process ownership of a state file.

- ServiceRegistry: pre-existing Go implementation plus corrected AuthorizationVerifyRequestDTO fields/operation names and removal of an implicit Sysop-name privilege escalation in outsourced token verification. Its original configuration and publisher are preserved.

Not implemented / not equivalent

1. MQTT APIs (since 2026-10-24) differ from Java in three deliberate points: the response topic must lie under arrowhead/response/<requester>/, requests with an invalid identity token are not answered (no 401 response), and identity login/logout/change stay HTTPS only. Request topics, envelopes, operation names and error payloads follow the Java MQTT API; see "Changes 2026-10-24 (MQTT)". 2. Dynamic federation, translation and QoS integration are absent. Requests that enable ALLOW_INTERCLOUD, ONLY_INTERCLOUD or ALLOW_TRANSLATION, or supply QoS requirements, fail explicitly with 501. They are not silently treated as successful local equivalents. Foreign-consumer management token generation also returns 501. This is unsuitable for a federated orchestration replacement until those paths are ported and tested. 3. Blacklist support and authorization-driven management filters are not ported to the four new systems. Management uses identity sysop privileges or an explicit whole-management whitelist. This is less granular than per-operation authorization; do not use the whitelist as a substitute for fine-grained policy enforcement. 4. The new systems do not yet publish native sysMonitor/AMQP telemetry. Their echo APIs and history/logs exist, but they will not satisfy a LocalCloudManager native-telemetry readiness requirement. The existing ServiceRegistry publisher remains available. 5. No Java SQL database compatibility/import. Each new system uses a private atomic JSON state file, not JPA/MySQL. Never mount a Java database volume as a Go state file. Java identity, authorization, key, store, lock, subscription and history data need a separately reviewed importer before migration. 6. Configuration is JSON, not Spring properties, environment override semantics, keystores or Java's full property catalog. get-config returns a sanitized native configuration subset in the Java map response wrapper. There is no cloud registration health/readiness gate; registration failures are retried and reported to the process log. 7. Validation/normalization and query semantics are not exhaustively matched. Supply canonical names and enum spelling. Field lengths, case normalization, HTTP error details/statuses, empty/null distinctions, ordering/default page sizes and warning text still need Java golden-contract tests. Pagination is bounded, but is not a full port of Java's per-entity paging service. JSON inputs use dynamic documents rather than generated, exhaustive DTO validation. 8. Metadata supports the documented comparisons and dotted/indexed paths, but REGEXP uses Go RE2, not Java Pattern; lookarounds/backreferences are not supported. Mixed-type numeric/collection edge cases need further Java fixture tests. 9. Dynamic candidate ranking is deterministic local/preferred ordering, not the complete Java matchmaking/QoS/fallback pipeline. Scopes are checked fail-closed; a denied requested operation can reject orchestration instead of yielding Java's precise partial-result behavior. Token acquisition is not rolled back if a later lock conflict aborts orchestration. 10. Push uses one worker and one delivery attempt, without Java's Quartz/thread-pool behavior, HTTP/plain MQTT callbacks, retries, or automatic event-driven re-orchestration. HTTPS result-delivery behavior is tested; duplicate delivery avoidance does not constitute exactly-once delivery. Notifications interrupted by a crash are reported as uncertain ERROR. 11. Logs are native bounded audit records (1,000 retained), not the full Java logging database or JVM-specific exceptions. Sessions/tokens/jobs/subscriptions/locks are validated for expiry at access but do not have complete Java cleaner/retention policies; state can grow over time. Expired entities may differ in query visibility. 12. No certificate revocation enforcement beyond Go's ordinary certificate-chain/time/hostname checks. No blacklist, rate limiter, distributed transactions, HA, distributed locks, SQL migration, or production security certification. Offline self-contained token revocation requires provider-side expiration/revocation handling; online revocation does not invalidate already-issued offline tokens.

Release gate

Do not describe this profile as fully interchangeable with the Java cloud. Before replacement: implement the required missing capabilities, compare successful and failing requests against Java for every DTO/operation, verify all discovery/security policies, migrate data with rollback, verify monitoring/UI readiness and federation, and rerun the cloud security evaluation. None of those deployment changes is made by this source delivery.

Changes 2026-10-17 (GoCloud deployment)

- Dynamic orchestration passes each requested operation as an authorization scope; a provider qualifies only if

every requested operation is granted. Without operations the all-scopes check applies (as before). Test: TestNativeServiceRegistryIntegration.

- Deployed in Arrowhead-test-cloud-go (see its GO-NATIVE-FULL-2026-10-17.md) by operator decision, with the gaps

listed above accepted; the release gate above is not thereby satisfied.

- Inter-cloud orchestration: ALLOW_INTERCLOUD (fallback when no local provider) and ONLY_INTERCLOUD through the local

Gatekeeper (gatekeeperURL): globalServiceDiscovery query, interCloudNegotiation negotiate, GateTunnel interface result. ALLOW_TRANSLATION and QoS requirements still return 501.

- Native sysMonitor telemetry (monitorEnabled, amqpURL): sysMonitor offer on generic_amqps, echo/power/status.

- managementPolicy: authorization: ConsumerAuthorization decides management access per service and operation.

- Requested operations match string-set operations (MQTT/AMQP templates) as well as HTTP operation maps.

- Push subscriptions store notifyRecipient: a non-sysop management subscriber (orchestration by proxy) receives

the result; own and sysop subscriptions deliver to the target.

Changes 2026-10-24 (translation)

- ALLOW_TRANSLATION: an interface is translatable when it does not match the template or property requirements

natively (as Java assortInterfacesAndMarkIfNonNative), so a bridge can also translate only the data model of the accepted template (generic_https senml -> generic_https simpleJson). Native candidates still win; the consumer's authorization for the provider is still checked before a bridge is built.

Changes 2026-10-24 (MQTT)

- MQTT API of all five core systems (internal/core/mqtt.go, ServiceRegistry/mqtt.go), the Java mqtt.api.enabled

counterpart: the Java request envelope (MqttRequestTemplate: traceId, authentication IDENTITY-TOKEN//<token>, responseTopic, qosRequirement, params, payload) on the Java base topics + operation (arrowhead/serviceregistry/{device-discovery,system-discovery,service-discovery,management,general/management, monitor}/, arrowhead/authentication/identity/[management/], arrowhead/consumer-authorization/authorization[-token]/ [management/], arrowhead/serviceorchestration/orchestration/[management/{push,history,lock,store,plant-rules}/], general/management/, monitor/echo); the response (MqttResponseTemplate: status, traceId, receiver, payload; errors as ErrorMessageDTO with exceptionType and origin = topic) goes to the request's response topic.

- The adapter verifies the identity token and runs the request in process through the HTTPS route, so the blacklist,

management (whitelist/authorization) checks and handlers are the same as over HTTPS. Path parameters come from params (a single one also from a string payload, as Java), the other params become the query, a list payload of a removal operation is its list parameter (names, instanceIds, uuids, ids, tokenReferences, systemNames, serviceInstances, keys).

- Response binding (Go): the response topic must lie under arrowhead/response/<system of the token>/; with a broker

ACL that lets each client read only its own prefix, a stolen token cannot be used to read answers.

- Configuration: mqttURL (ssl://host:port, client certificate = system certificate) and mqttTopicPrefix (a second

orchestrator on the same broker needs another prefix); ServiceRegistry mqtt.api.enabled, mqtt.broker.address, mqtt.broker.port, mqtt.topic.prefix (requires TLS and the outsourced policy). With MQTT enabled every service is also registered with a generic_mqtts interface (baseTopic, operation list), as Java.

- ServiceRegistry: the Blacklist filter never checks Blacklist itself (Java BlacklistFilter); a Java Blacklist, which

registers before it serves its check, was refused under force.blacklist.filter.

- cmd/mqtt-call: command-line client for one MQTT API request (tests).

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-core-rust/authentication.

Authentication (Rust)

Arrowhead 5.2.2 core system. src/main.rs is built as binary authentication by the arrowhead-core-rust crate in ... authentication.json and documentation/ link to ../examples and ../documentation/Authentication.

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-generalManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-generalManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identityManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identityManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-sysMonitor-AMQP-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-core-rust/documentation/Authentication/Authentication-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

generalManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the Authentication system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `generalManagement` service interface of the Authentication system, which provides general management of the Authentication system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8444 and base path `/authentication/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-log`	POST	`/logs`
`get-config`	GET	`/get-config`

2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /authentication/general/mgmt/logs

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /authentication/general/mgmt/logs HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

2.2 GET /authentication/general/mgmt/get-config?keys=*{keys}*

Operation: get-config
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /authentication/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.3 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.4 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.6 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-generalManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the Authentication system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct KeyValues . . . 6
3.5 struct LogEntry . . . 6
3.6 struct LogEntryListResponse . . . 7
3.7 struct LogRequest . . . 7
3.8 struct Page . . . 7
3.9 Primitives . . . 7

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `generalManagement` service interface of the Authentication system over MQTT, which provides general management of the Authentication system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/authentication/general/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`get-log`	`arrowhead/authentication/general/management/get-log`
`get-config`	`arrowhead/authentication/general/management/get-config`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/authentication/general/management/get-log`

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 3: Payload of an get-log request.

2.2 PUBLISH `arrowhead/authentication/general/management/get-config`

Operation: get-config
Input: Array<String>
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

```
[
  "string"
]
```

Listing 4: Payload of an get-config request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.5 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.6 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.8 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-generalManagement-SD.pdf.

[6] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (Authentication),” Eclipse Arrowhead, IDD 5.2.2, 2026, authentication-v5.2.2-generalManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

identity HTTP/TLS/JSON

Interface Design Description

Service ID: *"identity"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `identity` service of the Authentication system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation identity-login . . . 4
2.2 operation identity-logout . . . 4
2.3 operation identity-change-credentials . . . 5
2.4 operation identity-verify . . . 5

3 Data Models **7**
3.1 struct ErrorMessage . . . 7
3.2 struct IdentityChangeRequest . . . 7
3.3 struct IdentityLoginResponse . . . 7
3.4 struct IdentityRequest . . . 7
3.5 struct IdentityVerifyResponse . . . 7
3.6 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `identity` service interface of the Authentication system, which provides authentication of systems: a system logs in with its credentials and obtains an identity token, which it presents to other systems; providers verify such tokens.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `identity` service interface

This document provides the Interface Design Description IDD to the *identity – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `identity` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8444 and base path `/authentication/identity`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`identity-login`	POST	`/login`
`identity-logout`	POST	`/logout`
`identity-change-credentials`	POST	`/change`
`identity-verify`	GET	`/verify/{token}`

2 Service Operations

The interfaces of the `identity` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `identity` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /authentication/identity/login

Operation: identity-login
Input: IdentityRequest
Output: IdentityLoginResponse

Checks the credentials and opens a session. Returns the identity token and its expiry (`identity.token.duration`, 300 s in the default configuration). A previous token of the system becomes invalid.

A successful call returns `200 OK` with the output in the body.

```
POST /authentication/identity/login HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "systemName": "TemperatureSensor",
  "credentials": {
    "password": "<password>"
  }
}
```

Listing 1: An identity-login invocation.

```
{
  "token": "<token>",
  "expirationTime": "2026-10-22T12:00:00Z"
}
```

Listing 2: An identity-login response.

2.2 POST /authentication/identity/logout

Operation: identity-logout
Input: IdentityRequest

Checks the credentials and closes the session of the system.

A successful call returns `200 OK`.

```
POST /authentication/identity/logout HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json
```

```
5
6  {
7    "systemName": "TemperatureSensor",
8    "credentials": {
9      "password": "<password>"
10   }
11 }
```

Listing 3: An identity-logout invocation.

2.3 POST /authentication/identity/change

Operation: identity-change-credentials
Input: IdentityChangeRequest

Checks the current credentials and replaces them with the new ones.
A successful call returns `200 OK`.

```
1  POST /authentication/identity/change HTTP/1.1
2  Host: authentication:8444
3  Authorization: Bearer IDENTITY-TOKEN//<token>
4  Content-Type: application/json
5
6  {
7    "systemName": "TemperatureSensor",
8    "credentials": {
9      "password": "<password>"
10   },
11   "newCredentials": {
12     "password": "<password>"
13   }
14 }
```

Listing 4: An identity-change-credentials invocation.

2.4 GET /authentication/identity/verify/*{token}*

Operation: identity-verify
Input: String
Output: IdentityVerifyResponse

Checks an identity token. Returns whether it is valid and, if so, the system, its sysop flag, login and expiry time. The caller must itself present a valid identity token (only `login`, `logout` and `change` are token-free).
Path parameter: *token*.
A successful call returns `200 OK` with the output in the body.

```
1  GET /authentication/identity/verify/<token> HTTP/1.1
2  Host: authentication:8444
3  Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An identity-verify invocation.

```
1  {
2    "verified": true,
3    "systemName": "TemperatureSensor",
4    "sysop": false,
5    "loginTime": "2026-10-22T12:00:00Z",
6    "expirationTime": "2026-10-22T12:00:00Z"
7  }
```

Listing 6: An identity-verify response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.6, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct IdentityChangeRequest

Changes the credentials of the requesting system.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"credentials"	Object<String>	Credentials of the authentication method, for `PASSWORD` the key `password`.
"newCredentials"	Object<String>	The new credentials.

3.3 struct IdentityLoginResponse

The identity token issued at login.

Object Field	Value Type	Description
"token"	String	The identity token, used as `Bearer IDENTITY-TOKEN//<token>`.
"expirationTime"	DateTime	UTC time when the identity token expires.

3.4 struct IdentityRequest

System name and credentials.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"credentials"	Object<String>	Credentials of the authentication method, for `PASSWORD` the key `password`.

3.5 struct IdentityVerifyResponse

Result of verifying an identity token.

Object Field	Value Type	Description
"verified"	Boolean	True when the token is valid.
"systemName"	String	System the token belongs to.
"sysop"	Boolean	Whether that system is a system operator.
"loginTime"	DateTime	UTC time of the login.
"expirationTime"	DateTime	UTC time when the identity token expires.

3.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-identity-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

identity MQTT/TLS/JSON

Interface Design Description

Service ID: *"identity"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `identity` service of the Authentication system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation identity-login . . . 4
2.2 operation identity-logout . . . 4
2.3 operation identity-change-credentials . . . 5
2.4 operation identity-verify . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct IdentityChangeRequest . . . 6
3.5 struct IdentityLoginResponse . . . 6
3.6 struct IdentityRequest . . . 7
3.7 struct IdentityVerifyResponse . . . 7
3.8 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `identity` service interface of the Authentication system over MQTT, which provides authentication of systems: a system logs in with its credentials and obtains an identity token, which it presents to other systems; providers verify such tokens.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `identity` service interface

This document provides the Interface Design Description IDD to the *identity – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/authentication/identity/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`identity-login`	`arrowhead/authentication/identity/identity-login`
`identity-logout`	`arrowhead/authentication/identity/identity-logout`
`identity-change-credentials`	`arrowhead/authentication/identity/identity-change-credentials`
`identity-verify`	`arrowhead/authentication/identity/identity-verify`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/authentication/identity/identity-login`

Operation: identity-login
Input: IdentityRequest
Output: IdentityLoginResponse

Checks the credentials and opens a session. Returns the identity token and its expiry (`identity.token.duration`, 300 s in the default configuration). A previous token of the system becomes invalid.

```
{
  "systemName": "TemperatureSensor",
  "credentials": {
    "password": "<password>"
  }
}
```

Listing 3: Payload of an identity-login request.

2.2 PUBLISH arrowhead/authentication/identity/identity-logout

Operation: identity-logout
Input: IdentityRequest

Checks the credentials and closes the session of the system.

```
{
  "systemName": "TemperatureSensor",
  "credentials": {
    "password": "<password>"
  }
}
```

Listing 4: Payload of an identity-logout request.

2.3 PUBLISH arrowhead/authentication/identity/identity-change-credentials

Operation: identity-change-credentials
Input: IdentityChangeRequest

Checks the current credentials and replaces them with the new ones.

```
{
  "systemName": "TemperatureSensor",
  "credentials": {
    "password": "<password>"
  },
  "newCredentials": {
    "password": "<password>"
  }
}
```

Listing 5: Payload of an identity-change-credentials request.

2.4 PUBLISH arrowhead/authentication/identity/identity-verify

Operation: identity-verify
Input: String
Output: IdentityVerifyResponse

Checks an identity token. Returns whether it is valid and, if so, the system, its sysop flag, login and expiry time. The caller must itself present a valid identity token (only `login`, `logout` and `change` are token-free).

```
"<token>"
```

Listing 6: Payload of an identity-verify request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct IdentityChangeRequest

Changes the credentials of the requesting system.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"credentials"	Object<String>	Credentials of the authentication method, for `PASSWORD` the key `password`.
"newCredentials"	Object<String>	The new credentials.

3.5 struct IdentityLoginResponse

The identity token issued at login.

Object Field	Value Type	Description
"token"	String	The identity token, used as `Bearer IDENTITY-TOKEN//<token>`.
"expirationTime"	DateTime	UTC time when the identity token expires.

3.6 struct IdentityRequest

System name and credentials.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"credentials"	Object<String>	Credentials of the authentication method, for `PASSWORD` the key `password`.

3.7 struct IdentityVerifyResponse

Result of verifying an identity token.

Object Field	Value Type	Description
"verified"	Boolean	True when the token is valid.
"systemName"	String	System the token belongs to.
"sysop"	Boolean	Whether that system is a system operator.
"loginTime"	DateTime	UTC time of the login.
"expirationTime"	DateTime	UTC time when the identity token expires.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-identity-SD.pdf.

[6] ——, “identity HTTP/TLS/JSON – Interface Design Description (Authentication),” Eclipse Arrowhead, IDD 5.2.2, 2026, authentication-v5.2.2-identity-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

identityManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"identityManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `identityManagement` service of the Authentication system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation identity-mgmt-create . . . 4
2.2 operation identity-mgmt-update . . . 5
2.3 operation identity-mgmt-remove . . . 5
2.4 operation identity-mgmt-query . . . 5
2.5 operation identity-mgmt-session-close . . . 6
2.6 operation identity-mgmt-session-query . . . 7

3 Data Models **8**
3.1 struct ErrorMessage . . . 8
3.2 struct IdentityListMgmtCreateRequest . . . 8
3.3 struct IdentityListMgmtResponse . . . 8
3.4 struct IdentityListMgmtUpdateRequest . . . 8
3.5 struct IdentityMgmtRequest . . . 8
3.6 struct IdentityMgmtResponse . . . 8
3.7 struct IdentityQueryRequest . . . 9
3.8 struct IdentitySessionListMgmtResponse . . . 9
3.9 struct IdentitySessionQueryRequest . . . 9
3.10 struct IdentitySessionResponse . . . 9
3.11 struct Page . . . 10
3.12 Primitives . . . 10

4 References **11**

5 Revision History **12**
5.1 Amendments . . . 12
5.2 Quality Assurance . . . 12

1 Overview

This document describes the `identityManagement` service interface of the Authentication system, which provides operator management of the identities known to Authentication (systems, their credentials and sysop flag) and of their active sessions.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `identityManagement` service interface

This document provides the Interface Design Description IDD to the *identityManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `identityManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8444 and base path `/authentication/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`identity-mgmt-create`	POST	`/identities`
`identity-mgmt-update`	PUT	`/identities`
`identity-mgmt-remove`	DELETE	`/identities`
`identity-mgmt-query`	POST	`/identities/query`
`identity-mgmt-session-close`	DELETE	`/sessions`
`identity-mgmt-session-query`	POST	`/sessions`

2 Service Operations

The interfaces of the `identityManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `identityManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /authentication/mgmt/identities

Operation: identity-mgmt-create
Input: IdentityListMgmtCreateRequest
Output: IdentityListMgmtResponse

Creates identities with credentials of the given authentication method.

A successful call returns `200 OK` with the output in the body.

```
POST /authentication/mgmt/identities HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "authenticationMethod": "PASSWORD",
  "identities": [
    {
      "systemName": "TemperatureDisplay",
      "credentials": {
        "password": "<password>"
      },
      "sysop": false
    }
  ]
}
```

Listing 1: An identity-mgmt-create invocation.

```
{
  "identities": [
    {
      "systemName": "TemperatureSensor",
      "authenticationMethod": "PASSWORD",
      "sysop": false,
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedBy": "Sysop",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An identity-mgmt-create response.

2.2 PUT /authentication/mgmt/identities

Operation: identity-mgmt-update
Input: IdentityListMgmtUpdateRequest
Output: IdentityListMgmtResponse

Updates credentials and sysop flag of identities.
A successful call returns `200 OK` with the output in the body.

```
PUT /authentication/mgmt/identities HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "identities": [
    {
      "systemName": "TemperatureDisplay",
      "credentials": {
        "password": "<password>"
      },
      "sysop": false
    }
  ]
}
```

Listing 3: An identity-mgmt-update invocation.

```
{
  "identities": [
    {
      "systemName": "TemperatureSensor",
      "authenticationMethod": "PASSWORD",
      "sysop": false,
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedBy": "Sysop",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 4: An identity-mgmt-update response.

2.3 DELETE /authentication/mgmt/identities?names=*{names}*

Operation: identity-mgmt-remove
Input: Array<String>

Removes identities by system name, closing their sessions.
Query parameters: *names* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /authentication/mgmt/identities?names=<names> HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An identity-mgmt-remove invocation.

2.4 POST /authentication/mgmt/identities/query

Operation: identity-mgmt-query
Input: IdentityQueryRequest
Output: IdentityListMgmtResponse

Returns a page of identities matching the filter.
A successful call returns `200 OK` with the output in the body.

```
POST /authentication/mgmt/identities/query HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "namePart": "Temp",
  "isSysop": false,
  "createdBy": "Sysop",
  "creationFrom": "2026-10-22T12:00:00Z",
  "creationTo": "2026-10-22T12:00:00Z",
  "hasSession": true
}
```

Listing 6: An identity-mgmt-query invocation.

```
{
  "identities": [
    {
      "systemName": "TemperatureSensor",
      "authenticationMethod": "PASSWORD",
      "sysop": false,
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedBy": "Sysop",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 7: An identity-mgmt-query response.

2.5 DELETE /authentication/mgmt/sessions?names=*{names}*

Operation: identity-mgmt-session-close
Input: Array<String>

Closes the sessions of the named systems.
Query parameters: *names* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /authentication/mgmt/sessions?names=<names> HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 8: An identity-mgmt-session-close invocation.

2.6 POST /authentication/mgmt/sessions

Operation: identity-mgmt-session-query
Input: IdentitySessionQueryRequest
Output: IdentitySessionListMgmtResponse

Returns a page of active sessions.

A successful call returns `200 OK` with the output in the body.

```
POST /authentication/mgmt/sessions HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "namePart": "Temp",
  "loginFrom": "2026-10-22T12:00:00Z",
  "loginTo": "2026-10-22T12:00:00Z"
}
```

Listing 9: An identity-mgmt-session-query invocation.

```
{
  "sessions": [
    {
      "systemName": "TemperatureSensor",
      "loginTime": "2026-10-22T12:00:00Z",
      "expirationTime": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 10: An identity-mgmt-session-query response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.12, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct IdentityListMgmtCreateRequest

Identities to create.

Object Field	Value Type	Description
"authenticationMethod"	String	Authentication method of the identities; `PASSWORD` is the only implemented method.
"identities"	Array<IdentityMgmtRequest>	Identities to create, update or return.

3.3 struct IdentityListMgmtResponse

List of identities.

Object Field	Value Type	Description
"identities"	Array<IdentityMgmtResponse>	Identities to create, update or return.
"count"	Number	Total number of matching records (all pages).

3.4 struct IdentityListMgmtUpdateRequest

Identities to update.

Object Field	Value Type	Description
"identities"	Array<IdentityMgmtRequest>	Identities to create, update or return.

3.5 struct IdentityMgmtRequest

One identity.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"credentials"	Object<String>	Credentials of the authentication method, for `PASSWORD` the key `password`.
"sysop"	Boolean	Whether the identity is a system operator (default false).

3.6 struct IdentityMgmtResponse

A stored identity (without credentials).

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"authenticationMethod"	String	Authentication method of the identities; `PASSWORD` is the only implemented method.
"sysop"	Boolean	Whether the identity is a system operator (bypasses the management filters).
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.
"updatedBy"	String	Name of the system that changed the record last.
"updatedAt"	DateTime	UTC time of the last change.

3.7 struct IdentityQueryRequest

Paged filter for identities.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"namePart"	String	Part of the system name to match.
"isSysop"	Boolean	Only sysop (true) or non-sysop (false) identities.
"createdBy"	String	Name of the system that created the record.
"creationFrom"	DateTime	Lower bound of the creation time.
"creationTo"	DateTime	Upper bound of the creation time.
"hasSession"	Boolean	Only identities with (true) or without (false) an active session.

3.8 struct IdentitySessionListMgmtResponse

List of active sessions.

Object Field	Value Type	Description
"sessions"	Array<IdentitySessionResponse>	Active sessions.
"count"	Number	Total number of matching records (all pages).

3.9 struct IdentitySessionQueryRequest

Paged filter for active sessions.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"namePart"	String	Part of the system name to match.
"loginFrom"	DateTime	Lower bound of the login time.
"loginTo"	DateTime	Upper bound of the login time.

3.10 struct IdentitySessionResponse

An active session.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"loginTime"	DateTime	UTC time of the login.
"expirationTime"	DateTime	UTC time when the identity token expires.

3.11 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.12 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “identityManagement – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-identityManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

identityManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"identityManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `identityManagement` service of the Authentication system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation identity-mgmt-create . . . 4
2.2 operation identity-mgmt-update . . . 5
2.3 operation identity-mgmt-remove . . . 5
2.4 operation identity-mgmt-query . . . 5
2.5 operation identity-mgmt-session-close . . . 6
2.6 operation identity-mgmt-session-query . . . 6

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct IdentityListMgmtCreateRequest . . . 7
3.5 struct IdentityListMgmtResponse . . . 7
3.6 struct IdentityListMgmtUpdateRequest . . . 8
3.7 struct IdentityMgmtRequest . . . 8
3.8 struct IdentityMgmtResponse . . . 8
3.9 struct IdentityQueryRequest . . . 8
3.10 struct IdentitySessionListMgmtResponse . . . 9
3.11 struct IdentitySessionQueryRequest . . . 9
3.12 struct IdentitySessionResponse . . . 9
3.13 struct Page . . . 9
3.14 Primitives . . . 10

4 References **11**

5 Revision History **12**
5.1 Amendments . . . 12
5.2 Quality Assurance . . . 12

1 Overview

This document describes the `identityManagement` service interface of the Authentication system over MQTT, which provides operator management of the identities known to Authentication (systems, their credentials and sysop flag) and of their active sessions.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `identityManagement` service interface

This document provides the Interface Design Description IDD to the *identityManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/authentication/identity/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`identity-mgmt-create`	`arrowhead/authentication/identity/management/identity-mgmt-create`
`identity-mgmt-update`	`arrowhead/authentication/identity/management/identity-mgmt-update`
`identity-mgmt-remove`	`arrowhead/authentication/identity/management/identity-mgmt-remove`
`identity-mgmt-query`	`arrowhead/authentication/identity/management/identity-mgmt-query`
`identity-mgmt-session-close`	`arrowhead/authentication/identity/management/identity-mgmt-session-close`
`identity-mgmt-session-query`	`arrowhead/authentication/identity/management/identity-mgmt-session-query`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/authentication/identity/management/identity-mgmt-create`

Operation: identity-mgmt-create
Input: IdentityListMgmtCreateRequest
Output: IdentityListMgmtResponse

Creates identities with credentials of the given authentication method.

```
{
  "authenticationMethod": "PASSWORD",
```

```
  "identities": [
    {
      "systemName": "TemperatureDisplay",
      "credentials": {
        "password": "<password>"
      },
      "sysop": false
    }
  ]
}
```

Listing 3: Payload of an identity-mgmt-create request.

2.2 PUBLISH arrowhead/authentication/identity/management/identity-mgmt-update

Operation: identity-mgmt-update
Input: IdentityListMgmtUpdateRequest
Output: IdentityListMgmtResponse

Updates credentials and sysop flag of identities.

```
{
  "identities": [
    {
      "systemName": "TemperatureDisplay",
      "credentials": {
        "password": "<password>"
      },
      "sysop": false
    }
  ]
}
```

Listing 4: Payload of an identity-mgmt-update request.

2.3 PUBLISH arrowhead/authentication/identity/management/identity-mgmt-remove

Operation: identity-mgmt-remove
Input: Array<String>

Removes identities by system name, closing their sessions.

```
[
  "TemperatureSensor"
]
```

Listing 5: Payload of an identity-mgmt-remove request.

2.4 PUBLISH arrowhead/authentication/identity/management/identity-mgmt-query

Operation: identity-mgmt-query
Input: IdentityQueryRequest
Output: IdentityListMgmtResponse

Returns a page of identities matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "namePart": "Temp",
  "isSysop": false,
  "createdBy": "Sysop",
  "creationFrom": "2026-10-22T12:00:00Z",
  "creationTo": "2026-10-22T12:00:00Z",
  "hasSession": true
}
```

Listing 6: Payload of an identity-mgmt-query request.

2.5 PUBLISH arrowhead/authentication/identity/management/identity-mgmt-session-close

Operation: identity-mgmt-session-close
Input: Array<String>

Closes the sessions of the named systems.

```
[
  "TemperatureSensor"
]
```

Listing 7: Payload of an identity-mgmt-session-close request.

2.6 PUBLISH arrowhead/authentication/identity/management/identity-mgmt-session-query

Operation: identity-mgmt-session-query
Input: IdentitySessionQueryRequest
Output: IdentitySessionListMgmtResponse

Returns a page of active sessions.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "namePart": "Temp",
  "loginFrom": "2026-10-22T12:00:00Z",
  "loginTo": "2026-10-22T12:00:00Z"
}
```

Listing 8: Payload of an identity-mgmt-session-query request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct IdentityListMgmtCreateRequest

Identities to create.

Object Field	Value Type	Description
"authenticationMethod"	String	Authentication method of the identities; `PASSWORD` is the only implemented method.
"identities"	Array<IdentityMgmtRequest>	Identities to create, update or return.

3.5 struct IdentityListMgmtResponse

List of identities.

Object Field	Value Type	Description
"identities"	Array<IdentityMgmtResponse>	Identities to create, update or return.
"count"	Number	Total number of matching records (all pages).

3.6 struct IdentityListMgmtUpdateRequest

Identities to update.

Object Field	Value Type	Description
"identities"	Array<IdentityMgmtRequest>	Identities to create, update or return.

3.7 struct IdentityMgmtRequest

One identity.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"credentials"	Object<String>	Credentials of the authentication method, for `PASSWORD` the key `password`.
"sysop"	Boolean	Whether the identity is a system operator (default false).

3.8 struct IdentityMgmtResponse

A stored identity (without credentials).

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"authenticationMethod"	String	Authentication method of the identities; `PASSWORD` is the only implemented method.
"sysop"	Boolean	Whether the identity is a system operator (bypasses the management filters).
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.
"updatedBy"	String	Name of the system that changed the record last.
"updatedAt"	DateTime	UTC time of the last change.

3.9 struct IdentityQueryRequest

Paged filter for identities.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"namePart"	String	Part of the system name to match.
"isSysop"	Boolean	Only sysop (true) or non-sysop (false) identities.
"createdBy"	String	Name of the system that created the record.
"creationFrom"	DateTime	Lower bound of the creation time.
"creationTo"	DateTime	Upper bound of the creation time.
"hasSession"	Boolean	Only identities with (true) or without (false) an active session.

3.10 struct IdentitySessionListMgmtResponse

List of active sessions.

Object Field	Value Type	Description
"sessions"	Array<IdentitySessionResponse>	Active sessions.
"count"	Number	Total number of matching records (all pages).

3.11 struct IdentitySessionQueryRequest

Paged filter for active sessions.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"namePart"	String	Part of the system name to match.
"loginFrom"	DateTime	Lower bound of the login time.
"loginTo"	DateTime	Upper bound of the login time.

3.12 struct IdentitySessionResponse

An active session.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"loginTime"	DateTime	UTC time of the login.
"expirationTime"	DateTime	UTC time when the identity token expires.

3.13 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.14 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “identityManagement – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-identityManagement-SD.pdf.

[6] ——, “identityManagement HTTP/TLS/JSON – Interface Design Description (Authentication),” Eclipse Arrowhead, IDD 5.2.2, 2026, authentication-v5.2.2-identityManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the Authentication system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation echo . . . 4
2.2 operation power . . . 4
2.3 operation status . . . 4

3 Data Models **5**
3.1 struct Observation . . . 5
3.2 Primitives . . . 5

4 References **6**

5 Revision History **7**
5.1 Amendments . . . 7
5.2 Quality Assurance . . . 7

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of Authentication.
It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, Authentication registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.Authentication.Authentication",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher connects to the broker (`sysmonitor.broker.address`, default `amqp-broker`; port `sysmonitor.broker.port`, default 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. The local cloud name is taken from `sysmonitor.local.cloud` or the system certificate. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.Authentication.Authentication.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.Authentication.Authentication.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.Authentication.Authentication.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "Authentication-echo-1792670400000-42",
  "providerSystem": "Authentication",
  "serviceInstance": "Authentication",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "Authentication is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
}
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power` or `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

Mandatory mutual TLS, Arrowhead token/certificate binding, JSON request and response documents. Echo responses, orchestration subscription IDs, authorization public keys and encryption IV responses are text. Certificate-only operations and management are distinguished in the table.

1.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/authentication/general/mgmt/get-config	management
POST	/authentication/general/mgmt/logs	management
GET	/authentication/monitor/echo	certificate
POST	/authentication/identity/login	certificate
POST	/authentication/identity/logout	certificate
POST	/authentication/identity/change	certificate
GET	/authentication/identity/verify/{token}	identity
POST	/authentication/mgmt/identities	management
PUT	/authentication/mgmt/identities	management
DELETE	/authentication/mgmt/identities	management
POST	/authentication/mgmt/identities/query	management
DELETE	/authentication/mgmt/sessions	management
POST	/authentication/mgmt/sessions	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

2 Limitations

PASSWORD identities use bcrypt and single active opaque sessions. Java SQL/session import, complete query/time normalization and alternative framework transports require further work. No stored plaintext password or raw identity token is used.